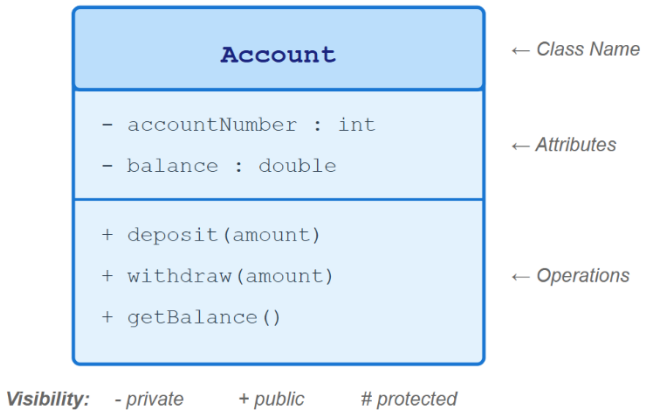
UML Class Diagram

A Class Diagram captures the static structure of an object-oriented system by describing the classes, their attributes, operations, and the relationships among them. As stated by Blaha & Rumbaugh, the class diagram forms the foundation of UML modelling, since all behavioral models are built on top of the static class structure.

- **Purpose of Class Diagrams**:
  - To identify the classes that exist in the system
  - To describe the attributes (state) and operations (behavior) of each class
  - To show relationships such as association, aggregation, composition, and inheritance
  - To provide a model that can be directly mapped to object-oriented programming languages
  - To support architectural design by defining system structure rather than behavior



Class Name

- Appears in the top compartment of the class diagram
- Represents the name of the class
- It is the only mandatory element of a UML class diagram

Attributes

- Represent the state (data) of an object
- Placed in the second compartment of the class
- Written in the format: attributeName : Type
- A derived attribute is computed from other attributes and is not stored explicitly
- Derived attributes are prefixed with /
- /age : int

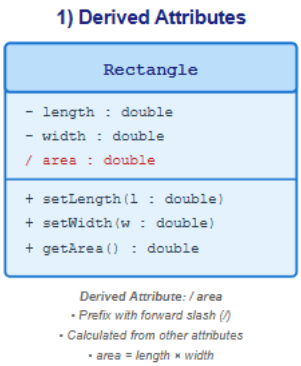
Operations

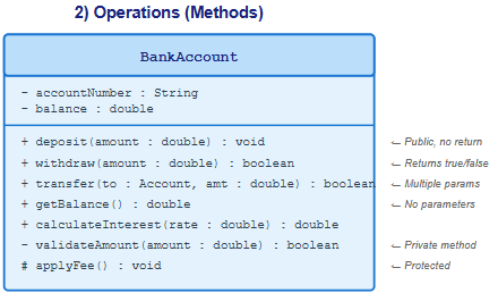
- Represent the behavior of the class
- Placed in the third compartment
- Defined using an operation signature, which includes:
  - Operation name

- Parameters (with types)
- Return type

Visibility Symbols

- + Public – accessible to any class
- - Private – accessible only within the class
- # Protected – accessible to subclasses





- **Approaches for identifying classes:**

1. Noun phrase approach
2. Common class patterns approach
3. Classes, responsibilities and collaborators (CRC) approach
4. Use case driven approach

1. **Noun phrase approach:**

- This approach was proposed by Rebecca, Brian and Lauren.
- The requirement document needs to be read and noun phrase should be identified.
- Nouns to be considered as classes and verbs to be methods of the classes.
- All plurals changed to singular.
- Nouns are listed and divided into three categories: relevant, fuzzy and irrelevant
- classes
- Irrelevant classes need to be identified.
- Candidate classes can be identified from other two categories.

➢ Identifying tentative classes:

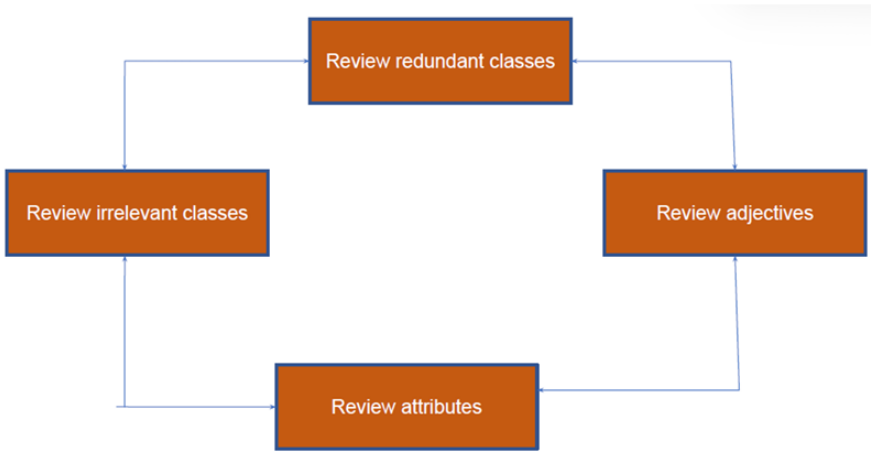
- Look for noun and noun phrases in the use cases.
- Some classes are implicit or taken from general knowledge.
- Carefully choose and define class names.
- Finding classes is an incremental and iterative process.

➢ Guidelines to select candidate classes from the relevant and fuzzy categories:

- Redundant classes: select one class if more than one class describes the same information. This is part of building a common vocabulary for the whole system. Choose the word that is used by the user.
- Attribute classes: Tentative objects that are used only as values should be defined or restarted as attributes and not as a class. For example, client status can be the attribute not the class.
- Irrelevant classes: class should be defined clearly with its purpose. If the purpose of statement is not possible for any class; class should be eliminated.

➢ The process of identifying classes and refining is the iterative process.

➢ This is not the sequential process. You can move back and forth.



2. Common Class Patterns Approach

- The Common Class Patterns Approach identifies candidate classes by using predefined, widely accepted patterns observed across many object-oriented systems.
- This approach was proposed by researchers such as Shlaer and Mellor.
- It focuses on recognizing recurring types of classes rather than extracting nouns directly from text.
- It is especially useful when domain knowledge is available and when noun-phrase analysis is insufficient or ambiguous.

➢ Types of Common Class Patterns:

1. **Concept Class**
   - Concept classes represent intangible principles or abstractions used to organize or track business activities.
   - They do not have a physical form but are central to the system logic.
   - Examples include:
     - Account
     - Customer
   - These classes often store rules, states, or identifiers rather than physical attributes.

2. **Event Class**
   - Event classes represent significant occurrences or happenings in the system.
   - They usually occur at a specific point in time or as part of a sequence.
   - Event classes are associated with information such as:
     - who
     - what
     - when
     - where
     - how
     - why
   - Examples include:
     - Transaction
     - Withdrawal
     - Deposit
   - These classes are important for auditing, tracking, and historical records.

3. **Organization Class**
   - Organization classes represent collections of people, resources, or facilities.
   - They describe entities to which users belong or with which the system is associated.
   - Examples include:
     - Bank
     - Branch
   - These classes often act as containers or aggregators for other classes.

4. **People Class**
   - People classes represent human roles relevant to the system.
   - They are divided into two categories:
     - Users who interact with the system, such as:
       - Operator
       - Clerk
       - RetailCustomer

        - CorporateCustomer
        - SeniorCitizenCustomer
    - People whose information is stored, but who do not directly use the system, such as:
        - Beneficiary
- These classes model roles, not just individuals.

5. **Place Class**
- Place classes represent physical locations about which the system must store information.
- They help model geographical or spatial aspects of the domain.
- Examples include:
    - Building
    - Store
    - Office
- These classes are commonly associated with organization or event classes.

6**. Tangible Things and Device Class**
- These classes represent physical objects or devices that the system interacts with.
- They have a tangible existence in the real world.
- Examples include:
    - Car
    - PressureSensor
- Device classes are especially important in embedded systems and real-time applications.